

JAG: Joint Attribute Graphs for Filtered Nearest Neighbor Search

Haike Xu¹ Guy Blelloch^{2,3} Laxman Dhulipala^{4,3} Lars Gottesbüren³ Rajesh Jayaram³ Jakub Łacki³

Abstract

Despite filtered nearest neighbor search being a fundamental task in modern vector search systems, the performance of existing algorithms is highly sensitive to query selectivity and filter type. In particular, existing solutions excel either at specific filter categories (e.g., label equality) or within narrow selectivity bands (e.g., pre-filtering for low selectivity) and are therefore a poor fit for practical deployments that demand generalization to new filter types and unknown query selectivities. In this paper, we propose JAG (Joint Attribute Graphs), a graph-based algorithm designed to deliver robust performance across the entire selectivity spectrum and support diverse filter types. Our key innovation is the introduction of *attribute* and *filter distances*, which transform binary filter constraints into continuous navigational guidance. By constructing a proximity graph that jointly optimizes for both vector similarity and attribute proximity, JAG prevents navigational dead-ends and allows JAG to consistently outperform prior graph-based filtered nearest neighbor search methods. Our experimental results across five datasets and four filter types (Label, Range, Subset, Boolean) demonstrate that JAG significantly outperforms existing state-of-the-art baselines in both throughput and recall robustness¹.

1. Introduction

Vector search has become a fundamental component of modern data management systems, driven by the increasing need to manage unstructured data and the wide popularity of deep learning embeddings. While search over raw vector similarity is well-studied, practical vector search queries rarely rely on similarity alone. Instead, users typically issue hybrid queries that combine high-dimensional vector similarity

¹Massachusetts Institute of Technology ²Carnegie Mellon University ³Google Research ⁴University of Maryland. Correspondence to: Haike Xu <haikexu@mit.edu>, Rajesh Jayaram <rkjaram@google.com>.

Preprint. February 12, 2026.

¹Our code is available at <https://github.com/xuhaike/JAG>

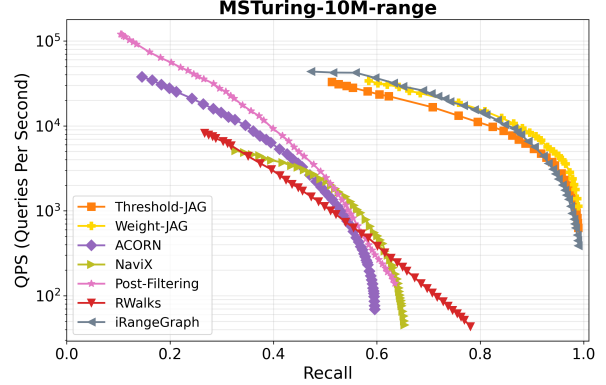


Figure 1. QPS vs. recall plot for range filters on the MSTuring-10M dataset. Please refer to Section 4 for experimental details.

with structured metadata constraints. For example, a user might query a product catalog to find items visually similar to a reference image (which is a vector search query using the image embedding) but restrict the search to a specific price range or category. This paradigm, broadly referred to as *filtered vector search*, introduces complex optimization trade-offs that differ significantly from standard relational query processing.

A critical parameter defining the performance landscape of filtered vector search is *selectivity*. We explicitly define selectivity as the fraction of items in the index that satisfy the filter condition. Consequently, a *high selectivity* query implies that a large portion of the dataset satisfies the filter, whereas a *low selectivity* query implies that only a small fraction of items are valid candidates. At the extreme ends of this selectivity spectrum, existing techniques provide robust solutions. In the high selectivity regime (where most items are valid), *post-filtering* is generally effective. This approach utilizes a standard, non-filtered vector search solution to retrieve a set of nearest neighbors, subsequently discarding those that fail the filter. Since the filter is permissive, the probability of finding sufficient valid neighbors within the top- k results remains high. Conversely, in the low selectivity regime (where very few items are valid), *pre-filtering* is the preferred strategy. Here, the system identifies the valid subset of data first—a task for which modern databases are heavily optimized—and subsequently performs an exact search on the reduced candidate set.

However, a significant challenge arises in the wide selectivity range between these two extremes. In this transition zone, the valid subset is too small for post-filtering to be efficient (requiring an excessively large scan of the vector index to find k valid results) yet too large for pre-filtering to be performant (as the "reduced" set may still contain too many items for brute-force calculation). Addressing this gap is an active area of study in the database community. A variety of solutions have been proposed, ranging from specialized graph traversals (Patel et al., 2024; Gollapudi et al., 2023; Ait Aomar et al., 2025; Wang et al., 2022; Li et al., 2025) to hybrid indexing structures (Gupta et al., 2023; Mohoney et al., 2023; Zuo et al., 2024). These existing approaches often exhibit distinct performance profiles: some are optimized specifically for certain filter types (e.g., label equality (Gollapudi et al., 2023), or range search (Zuo et al., 2024; Engels et al., 2024)), while others excel primarily within specific sub-ranges of selectivity.

To clarify the contributions of existing approaches and position our work, we classify hybrid search algorithms into three distinct categories based on their usage of metadata:

- (1) **Fully Oblivious (Filter Agnostic):** These methods (e.g., standard HNSW with post-filtering, ACORN (Patel et al., 2024), NaviX (Sehgal & Salihoglu, 2025)) build the index solely on vector data, ignoring attributes during construction. They are universally applicable but suffer performance degradation when filters are restrictive, as the index does not guide the search toward valid items.
- (2) **Filter-Aware:** These algorithms specialize the index structure for a specific class of query predicates known a priori, such as range queries (Zuo et al., 2024; Xu et al., 2024) or label equality or disjunction based filters (Gollapudi et al., 2023; Landrum et al., 2025; Cai et al., 2024). While they achieve good performance on their target query type, they are not robust to other types of queries: a range-optimized index cannot efficiently handle a Boolean or Subset query, requiring a different index for every filter type.
- (3) **Attribute-Aware (also Filter Agnostic):** These methods (Wang et al., 2022; Ait Aomar et al., 2025) use the attribute data distribution to organize the index during construction but do not hard-code the structure for a specific filter logic. However, existing methods are still primarily designed for binary match/non-match filters (e.g., label equality) and are not sufficiently general to support arbitrary filter types. In contrast, our goal is to support diverse filter types—including but not limited to label, range, subset, and Boolean filters—within a single unified index structure.

Specialized *filter-aware* algorithms will naturally outperform general-purpose methods on the specific task they are

optimized for. However, modern workloads are dynamic and diverse and maintaining separate indices for every possible filter type is often computationally prohibitive. Therefore, there is a strong incentive to design *filter-agnostic* algorithms which are competitive across a wide range of filter types and selectivity ranges.

In this paper, we propose a new filtered vector search algorithm designed to operate robustly across the selectivity spectrum and support a diverse array of filter types. Our algorithm is thus *filter-agnostic*, and is motivated by the observation that a given index configuration rarely suffices for all selectivity levels. Therefore, we introduce a method that effectively combines multiple index structures tuned for different selectivities into a single compact index. In particular, our algorithm builds a **Joint-Attribute Graph**, or a **JAG**, which is a *single* graph-based index that integrates multiple navigational layers corresponding to different query selectivities. To support this structural flexibility across broad filter types—including categorical, range, subset, and boolean filters—we formalize and utilize the concepts of *attribute distance* and *filter distance*.

The *attribute distance* ($dist_A$; used at build-time) measures the semantic proximity between the attributes of two data points (i.e., the metadata used to determine whether a data-point satisfies the filter), independent of any specific query. By constructing the JAG using a *joint* metric of the vector distance (the distance between two vectors) and attribute distances, we ensure connectivity even in regions where the vector space is sparse but attributes are similar. Crucially, we employ a "capped" attribute distance mechanism when building a JAG. By applying varying thresholds to $dist_A$, we generate a *hierarchy* of navigational edges: some connecting strict attribute neighbors (essential for low selectivity) and others connecting broader attribute neighborhoods (sufficient for high selectivity). This unified index structure in a JAG allows the search algorithm to seamlessly adapt its traversal strategy based on the selectivity of the query.

The *filter distance* ($dist_F$; used only at query-time when the filter is known) acts as a proxy for the binary filter constraint, quantifying how far a candidate's metadata is from satisfying the query. This prevents a query from hitting "dead ends" in the JAG by providing navigational proxy even when neighbors do not strictly satisfy the filter. Please refer to an example from Figure 2 to see how this proxy works.

Our experimental results using JAG show that our ideas yield significant improvements over prior state-of-the-art filtered vector search algorithms. For example, Figure 1 shows results on the MSTuring-10M dataset with range filters. All filter-agnostic algorithms reach at most 0.8 recall when their QPS is below 50, whereas our JAG achieves QPS > 10,000 at recall 0.8 and can still reach perfect recall with

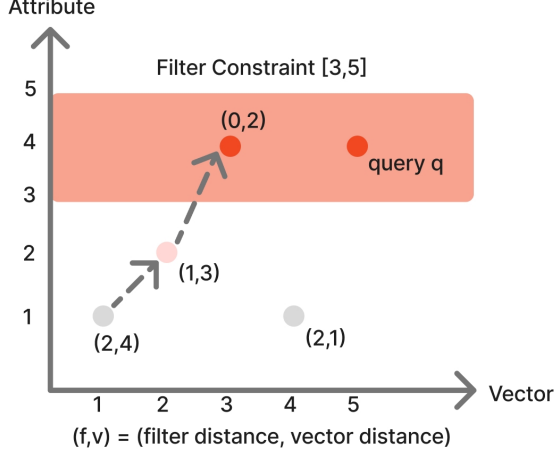


Figure 2. An example illustrating how JAG uses filter distance and vector distance to solve a range query. In this figure, both the vector value (x-axis) and attribute value (y-axis) are one-dimensional real numbers. The query specifies a filter range of $[3,5]$. The dashed arrow shows how filter distance and vector distance guide the greedy search toward the range query. Increasing intensity of the red color indicates improvement (decrease) in the filter distance.

$\text{QPS} > 500$. Furthermore, the performance of JAG matches that of iRangeGraph, an algorithm specifically designed for range filters.

Our specific contributions are as follows:

- (1) We propose a unified graph-based indexing that maintains robust performance across the selectivity spectrum by integrating edges optimized for varying filter strictness.
- (2) We formulate generalized attribute and filter distances that allow our index to support a wide range of constraints, including label, range, subset, and boolean logic, within a single framework.
- (3) We demonstrate the superior performance of our approach through an extensive experimental evaluation across five datasets. Compared to state-of-the-art baselines, our method consistently achieves the best performance in terms of query throughput and recall among all filter-agnostic algorithms. Crucially, on challenging and standard datasets with mixed query workloads (e.g., LAION-25M and YFCC), JAG achieves up to 10x higher QPS at recall 0.9 compared to the second-best filter-agnostic algorithm.

2. Preliminaries

Nearest Neighbors Search. We define the *nearest neighbor search* problem as follows. Let $X = \{x_1, \dots, x_n\}$ be a set of n points, where each $x \in \mathbb{R}^d$ is a d -dimensional vector. Given a query $x_q \in \mathbb{R}^d$, the goal of the nearest neighbor search is to design a data structure that can efficiently find $\arg \min_{x_p \in X} \text{dist}(x_p, x_q)$, where $\text{dist}(\cdot, \cdot)$ denotes a distance (or similarity) function. The quality of the returned candidate is typically evaluated by the *approximation ratio* (from a theoretical perspective) or *recall* (from an empirical perspective).

Filtered Nearest Neighbors Search. In the filtered setting, each point $p = (x_p, a_p)$ comes with an attribute $a_p \in \mathcal{A}$, and each query $q = (x_q, f_q)$ comes with a filter $f_q \in \mathcal{F}$. The search must be performed under the constraint that the point’s attribute satisfies the filter, i.e., $g(a_p, f_q) = 1$, where $g : \mathcal{A} \times \mathcal{F} \rightarrow \{0, 1\}$ is a binary matching function. Formally, the goal is to find $\arg \min_{p \in P: g(a_p, f_q)=1} \text{dist}(x_p, x_q)$

Filter Constraints.

Ideally, the filter constraint can be any function matching a pair of attribute and filter to pass / fail. We next specify several common forms of filter constraints considered in our work:

- (1) **Equality filter.** $\mathcal{A} = [L]$, $\mathcal{F} = [L]$, and

$$g(a, f) = \mathbf{1}[a = f].$$

This models categorical filtering, where each point belongs to exactly one category.

- (2) **Range filter.** $\mathcal{A} = \mathbb{R}$, $\mathcal{F} = \mathbb{R} \times \mathbb{R}$, and

$$g(a, f) = \mathbf{1}[f_{\min} \leq a \leq f_{\max}],$$

where $f = (f_{\min}, f_{\max})$. This captures numerical range filters.

- (3) **Subset filter.** $\mathcal{A} = \mathcal{F} = \{0, 1\}^L$, and

$$g(a, f) = \mathbf{1}[f \wedge a = f],$$

where $f \wedge a$ denotes elementwise AND. This models subset containment.

- (4) **Boolean filter.** $\mathcal{A} = \{0, 1\}^L$, $\mathcal{F}$ is a Boolean function over L variables, and

$$g(a, f) = \mathbf{1}[f(a) = 1].$$

This represents general logical filters, where the filter is an arbitrary boolean predicate evaluated on the attributes.

3. Joint Attribute Graphs (JAG)

In this section we detail the design and implementation of the Joint Attribute Graph (JAG), a unified indexing strategy capable of robust performance across varying selectivities. We first formalize the concepts of *filter distance* and *attribute distance*, which are essential for transforming binary filter constraints into continuous navigational gradients. We then explain how to integrate these metrics with standard vector distances using a capped attribute distance mechanism and a unified comparison rule to guide graph traversal. Finally, we present the specific algorithms for index

construction and search, introducing our primary method, Threshold-JAG, alongside a weighted variant, Weight-JAG.

3.1. Filter and Attribute Distances.

Unlike the binary constraint function $g : \mathcal{A} \times \mathcal{F} \rightarrow \{0, 1\}$ that only indicates whether an attribute-filter pair satisfies the constraint, we define two continuous distance functions:

(i) $dist_F : \mathcal{A} \times \mathcal{F} \rightarrow \mathbb{R}_{\geq 0}$, and (ii) $dist_A : \mathcal{A} \times \mathcal{A} \rightarrow \mathbb{R}_{\geq 0}$,

where $dist_F(f, a)$ measures how far an attribute a is from satisfying a filter f , and $dist_A(a_1, a_2)$ measures how different two attributes are with respect to satisfying a (unknown at indexing time) filter.

Definition. The function $dist_F$ must satisfy the following two properties:

- (1) **Validity** For any attribute $a \in \mathcal{A}$ and filter $f \in \mathcal{F}$

$$dist_F(a, f) = 0 \iff g(a, f) = 1$$

In other words, the filter distance is zero if and only if the attribute exactly satisfies the filter.

- (2) **Consistency.** For any filter $f \in \mathcal{F}$ and attributes $a_1, a_2 \in \mathcal{A}$,

$$dist_F(a_1, f) < dist_F(a_2, f)$$

is consistent with the heuristic interpretation that a_1 is closer to satisfying f than a_2 . $dist_F(\cdot, f)$ induces an ordering over attributes that correlates with proximity to filter satisfaction, even though satisfaction is discrete.

Similarly, the *attribute distance* $dist_A$ must satisfy two analogous criteria:

- (1) **Validity.** $dist_A(a_1, a_2) = 0$ if and only if $a_1 = a_2$.
 (2) **Consistency.** For any attributes $a_1, a_2, a_3 \in \mathcal{A}$,

$$dist_A(a_1, a_2) < dist_A(a_1, a_3)$$

is consistent with the heuristic interpretation that a_1 and a_2 are more likely to simultaneously pass or fail an unknown filter² than a_1 and a_3 . $dist_A$ induces an ordering over attribute pairs that correlates with similarity in filter outcomes.

These measures provide fine-grained relational information among attributes and filters, which can be exploited both when constructing proximity graphs over attributes and when routing queries through the graph.

Examples. The proposed distances can be naturally instantiated for several common filtering scenarios:

- (1) **Equality filter.** Attribute / filter represents a discrete

²This notion is somewhat informal, as the exact likelihood cannot be rigorously defined without knowing the filter in advance.

category. Let $\mathcal{A} = \mathcal{F} = [L]$. We define

$$dist_F(a, f) = \begin{cases} 0, & a = f, \\ 1, & a \neq f, \end{cases} \quad dist_A(a_1, a_2) = \mathbf{1}[a_1 \neq a_2].$$

- (2) **Range filter.** Attributes are scalar values (e.g., timestamps, prices). Let $\mathcal{A} = \mathbb{R}$ and $\mathcal{F} = \mathbb{R} \times \mathbb{R}$ with a filter $f = (f_{\min}, f_{\max})$. We define

$$dist_F(a, f) = \begin{cases} f_{\min} - a, & a < f_{\min}, \\ 0, & a \in [f_{\min}, f_{\max}], \\ a - f_{\max}, & a > f_{\max}, \end{cases}$$

$$dist_A(a_1, a_2) = |a_1 - a_2|,$$

where $dist_A$ measures numeric proximity between attributes.

- (3) **Subset filter.** Attributes encode set membership (e.g., tags, privileges). Let $\mathcal{A} = \mathcal{F} = \{0, 1\}^L$. We define

$$dist_F(a, f) = |f \setminus a|, \quad dist_A(a_1, a_2) = |a_1 \oplus a_2|,$$

where $\oplus$ denotes bitwise XOR. Here, $dist_F$ measures the number of elements a needs to cover f , while $dist_A$ measures the number of elements they differ.

- (4) **Boolean filter.** Attributes encode Boolean assignments over variables. Let $\mathcal{A} = \{0, 1\}^L$, and let $\mathcal{F}$ denote the set of all Boolean predicates. We define

$$dist_F(a, f) = \min_{a' : f(a')=1} |a - a'|,$$

$$dist_A(a_1, a_2) = |a_1 - a_2|,$$

where $|\cdot|$ denotes Hamming distance. Here, $dist_F(a, f)$ measures the minimum number of bit flips required to satisfy the Boolean predicate f .

Discussion. The specific choices of $dist_F$ and $dist_A$ are not unique and may vary across applications. In the worst case, one can always define trivial distances:

$$dist_F(a, f) = \mathbf{1}[g(f, a) = 0], \quad dist_A(a_1, a_2) = \mathbf{1}[a_1 \neq a_2]$$

which guarantees the feasibility of incorporating filter and attribute distances under any filtering scheme. Our algorithm is compatible with all such choices of $dist_F$ and $dist_A$, but the performance may vary depending on how informative the attribute/filter distance is.

3.2. Merging Attribute and Standard Distances

Given the definitions of attribute distance $dist_A$ and filter distance $dist_F$, a natural idea is to construct a proximity graph over attributes using $dist_A$, and then perform a greedy search guided by $dist_F$. Intuitively, this should allow us to quickly locate points whose attributes are most likely to satisfy the query filter. However, two major challenges remain:

- (1) **Inconsistency between attribute similarity and filter constraints.** We define attribute similarity as a proxy for approximating the likelihood that two attributes will

simultaneously pass an unknown filter. However, this approximation may be misleading if the index relies too heavily on attribute similarity. In the extreme case, an unfiltered query does not depend on attribute information at all. It is inherently difficult to approximate all unknown filters using a fixed attribute similarity.

- (2) **Combining attribute and standard proximity.** It is nontrivial to integrate the attribute/filter proximity graph with the standard proximity graph over the data vectors. Points that are close in the embedding space may have completely different attributes, and vice versa, making it unclear how to unify the two distance notions.

Prior work such as (Wang et al., 2022) and (Ait Aomar et al., 2025) has proposed constructing a proximity graph based on a weighted combination of the standard vector distance and the filter distance. While intuitive, this approach faces two limitations: (i) the two distances are often incomparable in scale, as one typically arises from the Euclidean space (e.g., ℓ_2 distance) while the other reflects discrete or categorical mismatches (e.g., keyword overlap); and (ii) the optimal weighting may depend on the query selectivity, which is unknown at construction time.

Our Solution. To address these challenges, we propose a new approach that introduces the concept of *capped attribute distance* and a unified comparison rule for candidate evaluation.

Capped attribute distance. We define a threshold-dependent variant of the attribute distance:

$$\text{dist}_A(a_1, a_2; t) = \max(\text{dist}_A(a_1, a_2) - t, 0),$$

where $t \geq 0$ is a tunable threshold. This formulation implies that once two attributes are sufficiently close (within distance t), they are treated as equivalent for indexing purposes. Conceptually, this introduces an “indicator-like” property: all attributes within the t -neighborhood are assumed to satisfy the “hypothetic” filter condition, while attributes farther away are penalized proportionally to their excess attribute distance.

Unified comparison rule. Given a base point $p = (x_p, a_p)$ and threshold t , we compare two candidate points $u = (x_u, a_u)$ and $v = (x_v, a_v)$ relative to p by first evaluating their capped attribute distances $\text{dist}_A(a_p, a_u; t)$ and $\text{dist}_A(a_p, a_v; t)$. If the two candidates are equally close in attribute distance, we break ties using their standard vector distances $\text{dist}(x_p, x_u)$ and $\text{dist}(x_p, x_v)$.

At query time, given a query q and two candidate points u and v , we analogously compare them using $\text{dist}_F(f_q, a_u)$ and $\text{dist}_F(f_q, a_v)$, and if equal, break ties using their standard vector distances to x_q .

This design unifies discrete filter constraints and continuous vector distances under a single graph-based framework, al-

lowing the search to adapt smoothly across different levels of query selectivity.

Formally, for any threshold parameter $t \geq 0$ and two points u and v , we define the unified distance as an ordered pair

$$D_A^t(u, v) = (\text{dist}_A(a_u, a_v; t), \text{dist}(x_u, x_v)),$$

Similarly, for a query q and a point u , we define

$$D_F(q, u) = (\text{dist}_F(f_q, a_u), \text{dist}(x_q, x_u)),$$

We compare $D_A^t(u, v)$ or $D_F(q, u)$ by lexicographic order.

3.3. Threshold-JAG

To support filters with varying selectivities, we construct proximity graphs using a set of thresholds T and perform greedy search over the resulting unified index. We refer to this version of our algorithm as Threshold-JAG.

Our index construction and query procedures rely on three components: **GreedySearch**, **Insert**, and **JointRobustPrune**. All procedures operate on a directed proximity graph $G = (P, E)$ and make use of customized comparators that incorporate both attribute distance and filter distance as we have discussed in Subsection 3.2.

GreedySearch (Algorithm 1): To find the nearest neighbors of a point x (which can be either a query or an existing data point) under a comparator D , the algorithm performs a greedy beam search on the graph starting from a fixed entry point s . At each iteration, it expands the closest unexplored vertex p , computes the comparator distances from x to all out-neighbors $N_{out}(p)$ of p , and adds these neighbors to a candidate list maintained as a priority queue ordered by $D(x, \cdot)$. The search terminates when all the top- l_s vertices in the queue have been explored. Finally, it returns the k closest explored vertices as the search result.

Query (Algorithm 2): To answer a filtered nearest-neighbor query (q, f) , we invoke **GreedySearch** using a comparator $D_F(f)$ that prioritizes candidates according to their filter distances dist_F (breaking ties by vector distance).

Insert (Algorithm 3): The graph is built incrementally. To insert a new point p with attribute a_p , we perform **GreedySearch** with comparator $D_A(t)$ for each threshold t in a predefined list T , and collect the union of all visited vertices V . The candidate set V is then pruned to a subset V' , $|V'| \leq R$, by invoking **JointRobustPrune**. Bidirectional edges are established between p and each $v \in V'$. If the degree of any vertex v exceeds R , we apply **JointRobustPrune** again to enforce the degree constraint.

JointRobustPrune (Algorithm 4): When a vertex exceeds the maximum out-degree R , this procedure selects a diverse subset of neighbors. Let T be the list of thresholds maintained in the index and deg the total degree budget. We partition the degree budget into $|T|$ buckets, assigning

Algorithm 1 GreedySearch(G, q, k, l_s, D)

```

1: Input: Graph  $G = (P, E)$ , query point  $q$ , beam size  $l_s$ ,
   comparator  $D$ 
2: Output: Top- $k$  nearest vertices to  $q$ , visited set  $V$ 
3: Initialize candidate list  $L \leftarrow \{s\}$  ( $s$  is the entry vertex)
4: Initialize visited set  $V \leftarrow \emptyset$ 
5: while  $L \setminus V \neq \emptyset$  do
6:    $p \leftarrow \arg \min_{v \in L \setminus V} D(q, v)$ 
7:    $V \leftarrow V \cup \{p\}$ 
8:   for each  $u \in N_{\text{out}}(p)$  do
9:     if  $u \notin L$  then
10:       $L \leftarrow L \cup \{u\}$ 
11:     end if
12:   end for
13:   if  $|L| > l_s$  then
14:     Retain the top- $l_s$  vertices in  $L$  ranked by  $D(q, \cdot)$ 
15:   end if
16: end while
17: return Top- $k$  vertices in  $V$  ranked by  $D(q, \cdot)$ , and  $V$ 
    
```

Algorithm 2 Query(G, q, l_s, k)

```

1: Input: Graph  $G = (P, E)$ , query point  $q$ , search beam
   size  $l_s$ , number of returned neighbors  $k$ 
2: Output: Top- $k$  nearest vertices to  $q$ 
3:  $[A, V] \leftarrow (\text{GreedySearch}(G, q, k, l_s, D_F))$ 
4: return  $A$ 
    
```

deg $/ |T|$ neighbors per threshold. For each $t \in T$, we sort candidates by comparator $D_A(t)$, then iterate through the sorted list in order: if a candidate v is not dominated by previously selected vertices, we include it in V' and prune later vertices v' for which $\text{dist}_V(v, v') < \text{dist}_V(p, v')/\alpha$ according to the standard RobustPruning criteria in (Subramanya et al., 2019). The process continues until each bucket reaches its local degree limit deg $/ |T|$. Finally, we merge all buckets to form the new neighbor list V' , ensuring $|V'| \leq \text{deg}$.

3.4. Weight-JAG

We also implement a weighted variant of JAG, in which attribute distance and vector distance are combined using different weights. For a weight w , we define

$$D_A^w(u, v) = w \cdot \text{dist}_A(a_u, a_v) + \text{dist}(x_u, x_v).$$

We build the index graph using a set of such weights. Similar to Threshold-JAG, when calling Insert and JointRobustPrune, we iterate over the weight list and use $D_A^w(u, v)$ as the comparator. During query processing, we still compare $\text{dist}_F(\cdot)$ first, followed by the standard vector distance. We refer to this variant as Weight-JAG. Please refer to Appendix C for experimental comparisons.

Algorithm 3 Insert(G, p, T, l_b, R, α)

```

1: Input: Graph  $G = (P, E)$ , new point  $p$ , threshold
   list  $T$ , build beam size  $l_b$ , degree bound  $R$ , pruning
   parameter  $\alpha$ 
2: Output: Updated graph  $G' = (P', E')$ 
3: Initialize  $V \leftarrow \emptyset$ 
4: for each  $t \in T$  do
5:    $[A_t, V_t] \leftarrow \text{GreedySearch}(G, p, 1, l_b, D_A(t))$ 
6:    $V \leftarrow V \cup V_t$ 
7: end for
8:  $N_{\text{out}}(p) \leftarrow \text{JointRobustPrune}(G, p, V, R, \alpha, T)$ 
9: for each  $v \in N_{\text{out}}(p)$  do
10:   $N_{\text{out}}(v) \leftarrow N_{\text{out}}(v) \cup \{p\}$ 
11:  if  $|N_{\text{out}}(v)| > R$  then
12:     $N_{\text{out}}(v) \leftarrow \text{JointRobustPrune}(G, v, N_{\text{out}}(v), R, \alpha, T)$ 
13:  end if
14: end for
15: return  $G'$ 
    
```

4. Experiments

We compare our algorithm, JAG, against ten baseline algorithms on five datasets. Please see our code in the supplementary material. We summarize our findings below.

- Across all datasets (SIFT, ARXIV, LAION, YFCC, and MSTuring), JAG consistently outperforms all filter-agnostic algorithms (see Figure 1, 3, 5, 4).
- For datasets with low-selectivity queries (e.g., selectivity $< 1/100$ on MSTuring), JAG is the only filter-agnostic method that achieves perfect recall. All other filter-agnostic algorithms plateau below 0.8 recall, even when operating at extremely low throughput (QPS < 50), whereas JAG attains QPS > 1000 at 0.8 recall (see Figure 1, 5, 4).
- JAG exhibits the best QPS performance as the data scale increases (see Figure 7).
- JAG achieves the highest QPS when there is correlation between the query filter and the vector space (see Figure 6).

4.1. Datasets

We evaluate our algorithms on five datasets under various filter setups, including **SIFT**, **ARXIV**, **LAION**, **YFCC**, and **MSTuring**, each paired with one or more of the *Label*, *Range*, *Subset*, and *Boolean* filter types. We summarize the dataset size, average selectivity in Table 1. Please refer to Appendix D.2 for details on all datasets.

4.2. Algorithms

We implement both Threshold-JAG and Weight-JAG in our experiment. Please refer to Section D.3 for implementation details. We compare our algorithm against the following

Algorithm 4 JointRobustPrune(G, p, V, R, α, T)

```

1: Input: Graph  $G = (P, E)$ , vertex  $p$ , candidate set  $V$ ,
   total degree budget  $R$ , pruning parameter  $\alpha$ , threshold
   list  $T$ 
2: Output: Pruned neighbor list  $V'$ 
3: Initialize  $V' \leftarrow \emptyset$ 
4: for each  $t \in T$  do
5:   Initialize  $V'_t \leftarrow \emptyset$ 
6:   Sort  $V$  in increasing order of  $D_A(t)$ 
7:   for each  $v \in V$  (in order) do
8:     if  $\forall u \in V'_t, \alpha \cdot \text{dist}(u, v) > \text{dist}(p, v)$  then
9:        $V'_t \leftarrow V'_t \cup \{v\}$ 
10:    end if
11:    if  $|V'_t| \geq R/|T|$  then
12:      break
13:    end if
14:  end for
15:   $V' \leftarrow V' \cup V'_t$ 
16: end for
17: return  $V'$ 
    
```

baseline algorithms.

Baselines. We compare JAG with a comprehensive set of baseline algorithms, including ACORN (Patel et al., 2024), NaviX (Sehgal & Salihoglu, 2025), RWalks (Ait Aomar et al., 2025), Post-Filtering. We also test some other baseline algorithms, which only support certain filters. For example, we test FilteredVamana (Gollapudi et al., 2023), StitchedVamana (Gollapudi et al., 2023), and UNG (Cai et al., 2024) on Label and Subset filters, NHQ (Wang et al., 2022) on Label filters, and iRangeGraph (Xu et al., 2024) on Range Filters. We summarize their compatibility in Table 2. We describe each baseline algorithm in Appendix D.4 and their parameter choices in Appendix D.5.

4.3. Results

Please see our QPS v.s. recall plots in Figure 3, 5, 4, distance computation v.s. recall plots in Figure 10, 11, 13 in Appendix D.7, and indexing time in Table 3. For the Pre-Filtering algorithm, it always achieves perfect recall but its QPS is usually too low to show on the plots. We report its QPS and the number of distance computations it performs in Table 1. In the following, we elaborate on the algorithm analysis for different filters.

Label filters. Figure 3 shows the performance of all algorithms on the SIFT1M and ARXIV datasets with label filters. For the SIFT dataset, all algorithms achieve perfect recall, though with different QPS values. Our algorithm is slower than those filter specific algorithms like StitchedVamana, FilteredVamana, UNG, but outperforms all other methods. This is expected, as for non-overlapping label filters, the optimal solution is to build a separate index for each label

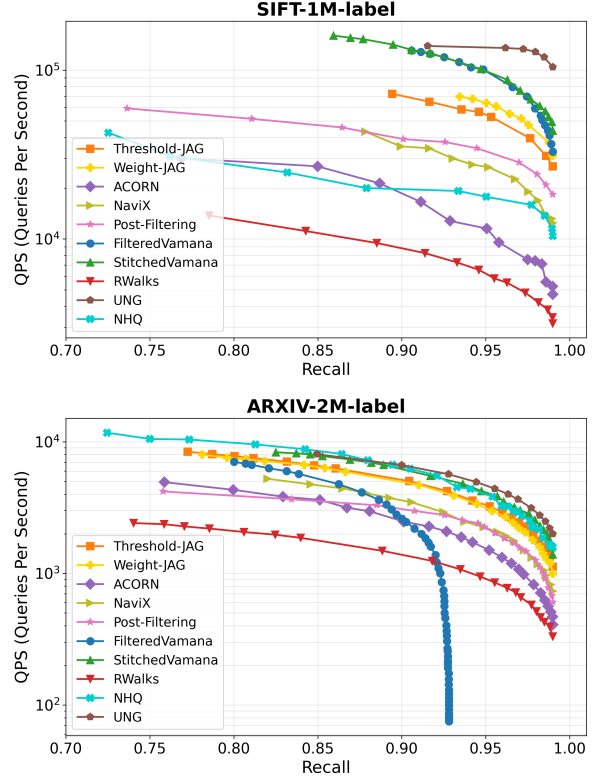


Figure 3. QPS vs. recall plot for Label filters on the SIFT and ARXIV datasets. Note that NHQ is designed specifically for Label filter. FilteredVamana, StitchedVamana, and UNG are designed specifically for Label and Subset filters.

and then perform standard nearest neighbor search within the subset of points sharing that label — which is essentially what StitchedVamana, FilteredVamana, and UNG are doing for such cases. On the ARXIV dataset, our algorithm, along with StitchedVamana, FilteredVamana, and UNG, performs almost identically and better than all other algorithms.

Range filters. Figure 1, and upper Figure 5 show the performance of all algorithms on the ARXIV and MSTuring datasets with range filters. For the ARXIV dataset, all algorithms achieve perfect recall, with our algorithm attaining the highest QPS than all the other filter-agnostic algorithms, but less efficient than iRangeGraph, which is an algorithm specially designed for range queries. On the MSTuring dataset, it is noteworthy that only our algorithm and iRangeGraph are able to reach perfect recall, while all other general purpose algorithms achieve at most 0.8 recall. The reason is that our synthetic range filters contain filter ranges with varying selectivity; we hypothesize that the other algorithms cannot effectively handle highly selective cases. Please refer to our ablation studies in Figure 8 for a more detailed analysis.

Subset filters. Figure 4 shows the performance of all algorithms on the MSTuring, YFCC, and LAION datasets

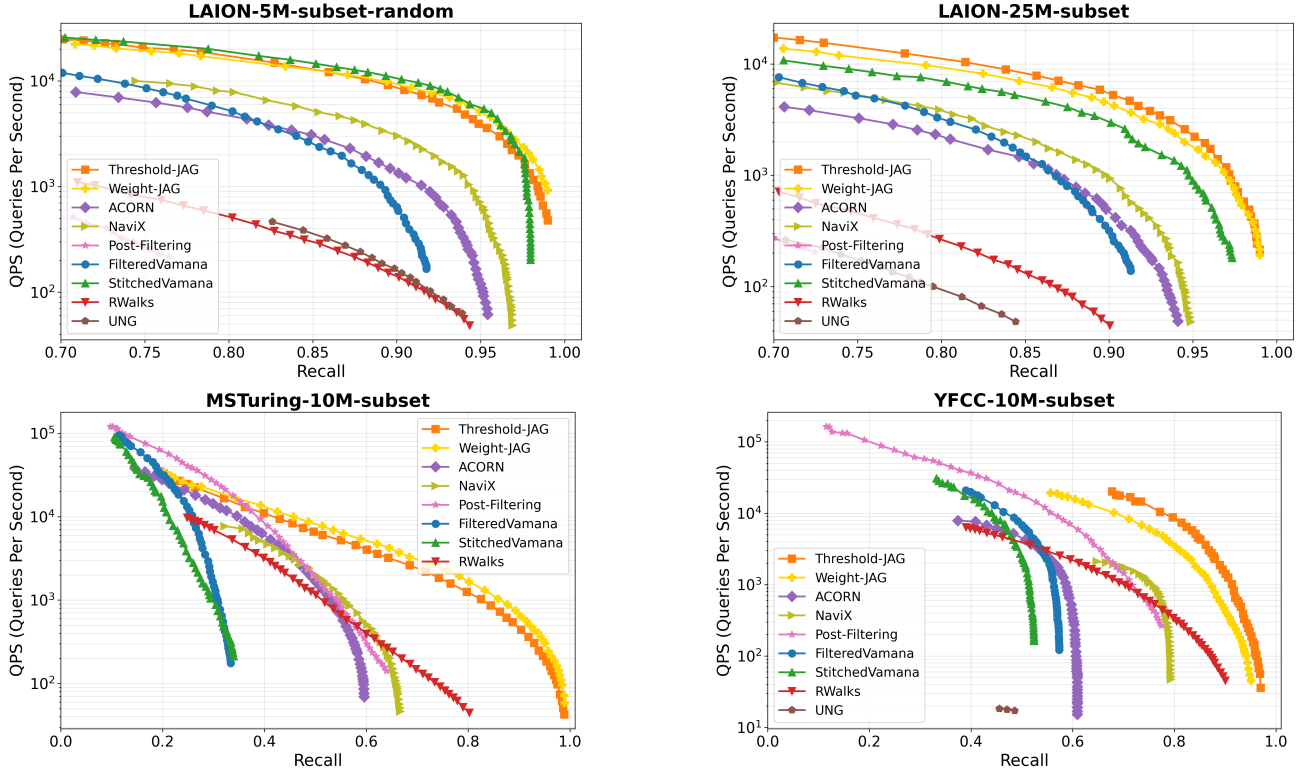


Figure 4. QPS vs. recall plots for subset filters on the MSTuring-10M, LAION-5M, and 25M datasets, and for boolean filters on the MSTuring-10M dataset. Note that FilteredVamana and StitchedVamana are only for label and subset filters.

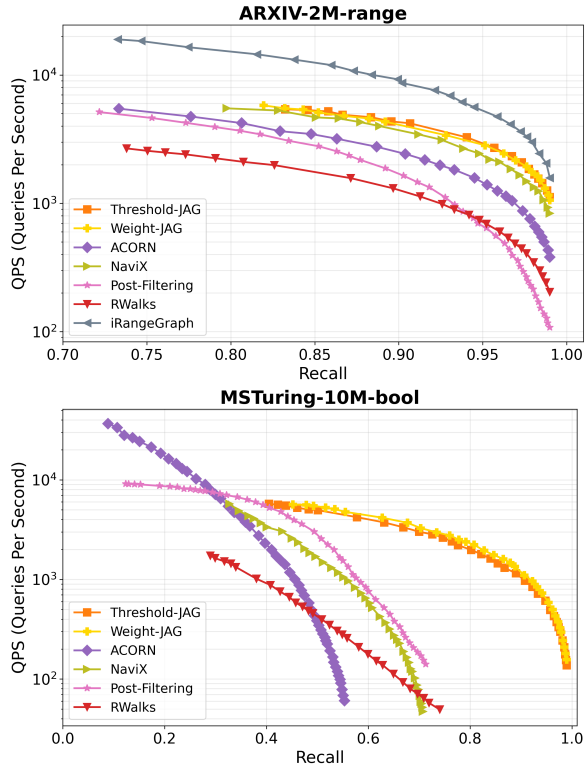


Figure 5. QPS vs. Recall plot for range filters on the ARXIV and bool filters on MSTuring datasets. Note that iRange-Graph is designed specifically for Range filter

with subset filters. For the MSTuring and YFCC datasets, only our algorithm is able to achieve perfect recall, while all other algorithms reach at most 0.8 on MSTuring and 0.9 on YFCC before their QPS drops below 50. As noted in (Ait Aomar et al., 2025), most queries from the YFCC dataset are extremely selective ($<1\%$), where no previous graph-based algorithm has achieved recall greater than 0.9 without using auxiliary IVF structures or pre-filtering. Here, we are the first graph-based algorithm to reach almost perfect recall (>0.95) with $\text{QPS} > 50$. Our algorithm performs comparably to StitchedVamana on LAION-5M, and our advantage becomes larger as the dataset size goes to 25M.

Boolean filters. Lower figure 5 shows the performance of all algorithms on the MSTuring dataset with boolean filters. Only our algorithm is able to achieve perfect recall, while all other algorithms reach at most 0.8 on MSTuring before their QPS drops below 50.

5. Conclusion

In this paper, we introduced JAG (Joint Attribute Graphs), a unified graph-based indexing strategy designed for filtered nearest neighbor search. Key to JAG’s robustness across different filter types and selectivities are the new notions of attribute distance, filter distance, and capped attribute distances, which enable the use of thresholded distance functions to provide navigational guidance and prevent navi-

gational dead-ends. Unlike prior methods that are restricted to specific filter types or struggle to achieve good performance across a broad range of the selectivity regime, JAG unifies edges tailored to different distance thresholds during construction and as a result is capable of handling filters across a broad selectivity range.

Impact Statement

This paper presents work whose goal is to advance the field of machine learning. There are many potential societal consequences of our work, none of which we feel must be specifically highlighted here.

References

- Ait Aomar, A., Echihabi, K., Arnaboldi, M., Alagiannis, I., Hilloulou, D., and Cherkaoui, M. Rwalks: Random walks as attribute diffusers for filtered vector search. *Proceedings of the ACM on Management of Data*, 3(3):1–26, 2025.
- Cai, Y., Shi, J., Chen, Y., and Zheng, W. Navigating labels and vectors: A unified approach to filtered approximate nearest neighbor search. *Proceedings of the ACM on Management of Data*, 2(6):1–27, 2024.
- Engels, J., Landrum, B., Yu, S., Dhulipala, L., and Shun, J. Approximate nearest neighbor search with window filters. In *International Conference on Machine Learning*, pp. 12469–12490. PMLR, 2024.
- Gollapudi, S., Karia, N., Sivashankar, V., Krishnaswamy, R., Begwani, N., Raz, S., Lin, Y., Zhang, Y., Mahapatro, N., Srinivasan, P., et al. Filtered-diskann: Graph algorithms for approximate nearest neighbor search with filters. In *Proceedings of the ACM Web Conference 2023*, pp. 3406–3416, 2023.
- Gupta, G., Yi, J., Coleman, B., Luo, C., Lakshman, V., and Shrivastava, A. Caps: A practical partition index for filtered similarity search. *arXiv preprint arXiv:2308.15014*, 2023.
- Iff, P., Brügger, P., Chrapek, M., Besta, M., and Hoefler, T. Benchmarking filtered approximate nearest neighbor search algorithms on transformer-based embedding vectors. *arXiv preprint arXiv:2507.21989*, 2025.
- Jegou, H., Douze, M., and Schmid, C. Hamming embedding and weak geometric consistency for large scale image search. In *European conference on computer vision*, pp. 304–317. Springer, 2008.
- Landrum, B., Manohar, M. D., Karjekar, M., and Dhulipala, L. ivf^2 index: Fusing classic and spatial inverted indices for fast filtered anns. In *The 1st Workshop on Vector Databases*, 2025.
- Li, Z., Huang, S., Ding, W., Park, Y., and Chen, J. Sieve: Effective filtered vector search with collection of indexes. *arXiv preprint arXiv:2507.11907*, 2025.
- Lowe, D. G. Distinctive image features from scale-invariant keypoints. *International journal of computer vision*, 60(2):91–110, 2004.
- Malkov, Y. A. and Yashunin, D. A. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *CoRR*, abs/1603.09320, 2016. URL <http://arxiv.org/abs/1603.09320>.
- Mohoney, J., Pacaci, A., Chowdhury, S. R., Mousavi, A., Ilyas, I. F., Minhas, U. F., Pound, J., and Rekatsinas, T. High-throughput vector similarity search in knowledge graphs. *Proceedings of the ACM on Management of Data*, 1(2):1–25, 2023.
- Patel, L., Kraft, P., Guestrin, C., and Zaharia, M. Acorn: Performant and predicate-agnostic search over vector embeddings and structured data. *Proc. ACM Manag. Data*, 2(3), May 2024. doi: 10.1145/3654923. URL <https://doi.org/10.1145/3654923>.
- Sehgal, G. and Salihoglu, S. Navix: A native vector index design for graph dbmss with robust predicate-agnostic search performance. *arXiv preprint arXiv:2506.23397*, 2025.
- Subramanya, S. J., Devvrit, F., Kadekodi, R., Krishnawamy, R., and Simhadri, H. V. Diskann: Fast accurate billion-point nearest neighbor search on a single node. In Wallach, H. M., Larochelle, H., Beygelzimer, A., d’Alché-Buc, F., Fox, E. B., and Garnett, R. (eds.), *Advances in Neural Information Processing Systems 32: Annual Conference on Neural Information Processing Systems 2019, NeurIPS 2019, 8-14 December 2019, Vancouver, BC, Canada*, pp. 13748–13758, 2019. URL <https://dl.acm.org/doi/abs/10.5555/3454287.3455520>.
- Thomee, B., Shamma, D. A., Friedland, G., Elizalde, B., Ni, K., Poland, D., Borth, D., and Li, L.-J. Yfcc100m: The new data in multimedia research. *Communications of the ACM*, 59(2):64–73, 2016.
- Wang, M., Lv, L., Xu, X., Wang, Y., Yue, Q., and Ni, J. Navigable proximity graph-driven native hybrid queries with structured and unstructured constraints. *arXiv preprint arXiv:2203.13601*, 2022.
- Xu, Y., Gao, J., Gou, Y., Long, C., and Jensen, C. S. irange-graph: Improvising range-dedicated graphs for range-filtering nearest neighbor search. *Proceedings of the ACM on Management of Data*, 2(6):1–26, 2024.

Zhang, D., Li, J., Zeng, Z., and Wang, F. Jasper and stella: distillation of sota embedding models. *arXiv preprint arXiv:2412.19048*, 2024.

Zuo, C., Qiao, M., Zhou, W., Li, F., and Deng, D. Serf: segment graph for range-filtering approximate nearest neighbor search. *Proceedings of the ACM on Management of Data*, 2(1):1–26, 2024.

A. Related Work

One solution to filtered vector search is to simply leverage the database to handle the filter. *Pre-filtering* first isolates the subset of points satisfying the metadata constraints (e.g., using a SQL query) and subsequently computes distances within this reduced candidate set. While this method guarantees correctness, it becomes computationally expensive in high-selectivity regimes where the filtered subset remains large.

Fully-Oblivious Methods. We define fully-oblivious methods as those that ignore attributes at index construction time. *Post-filtering* generates an initial list of nearest neighbors using a standard unfiltered index (e.g., HNSW (Malkov & Yashunin, 2016) or Vamana (Subramanya et al., 2019)) and retrospectively discards items that fail the filter criteria. This approach is effective when filters are permissive (high selectivity) but often fails to retrieve sufficient valid neighbors when the valid items are sparse or distant from the query in the vector space.

To mitigate the recall issues of standard post-filtering, more advanced fully-oblivious approaches such as ACORN (Patel et al., 2024) and NaviX (Sehgal & Salihoglu, 2025) utilize indexes constructed solely from vector data but modify the search procedure. These methods typically employ expanded neighborhood strategies—such as two-hop traversal—to explore a larger region of the vector space, aiming to encounter a sufficient number of valid candidates. However, these approaches implicitly assume that points satisfying the filter are uniformly distributed within the vector neighborhood. Consequently, performance degrades significantly when the filter is negatively correlated with vector distance or when selectivity is low.

Filter-Aware Methods. Filter-aware algorithms tailor the index structure to specific classes of query predicates known a priori. For example, FilteredVamana and StitchedVamana (Gollapudi et al., 2023) modify the DiskANN architecture to accommodate label constraints: the former restricts navigation to valid points, while the latter merges separate graphs constructed for individual filters. Similarly, other specialized methods augment proximity graphs with auxiliary data structures, such as tries or segment trees, to handle specific filter types. While these solutions excel at their targeted tasks (e.g., label equality or range search), they lack generalization; optimizing for one filter type often precludes efficient handling of others, such as Boolean or Subset queries.

Attribute-Aware (Filter-Agnostic) Methods. Attribute aware (or filter-agnostic) methods leverage the attributes at index construction time to build indexes that are optimized for many (unknown) queries. There are relatively few methods that can be categorized as attribute aware. NHQ (Wang et al., 2022) is an early filtered search algorithm that defines an attribute distance between two points based solely on whether their attributes are equal, and combines this with the standard vector distance using a weighted average. As a result, NHQ can only handle equality-based filters (and thus it could fairly be classified as a filter-aware method). The only other attribute-aware method, to the best of our knowledge, is a recent work called RWalks (Ait Aomar et al., 2025) which uses a standard (unfiltered) index and augments it for filtered search. At index construction time, the algorithm performs a random walk on the unfiltered index to compute an aggregated attribute vector. It then defines the filter distance as the fraction of labels in this vector that match the query filter. At query time, this filter distance is combined with the usual vector distance using a weighted scoring function to guide greedy search.

B. Additional Experimental Results

Scaling Experiment. For the LAION dataset, we select the three best algorithms—ours, ACORN, and StitchedVamana—and run them on datasets of different sizes (1M, 5M, and 25M) to evaluate their scaling behavior. Please see Figure 7 for the results. Our algorithm performs comparable with StitchedVamana on 1M and 5M scales, and then our advantage becomes more obvious in the 25M scale. All the QPS curves move downward as the dataset size increases from 1M to 5M to 25M, while our algorithm continues to achieve perfect recall and maintains $\text{QPS} > 50$.

Correlation. We test how the algorithms perform when the query filter is spatially correlated with the vectors. For the LAION-5M dataset, each keyword represents a point cluster in the vector space. To create a positive correlation between the query and the filter, we set the query filter to be the closest keyword cluster to the query image embedding. To create a negative correlation, we pick the query filter to be the farthest keyword cluster from the query image embedding. Please see Figure 6 for the results. For the positive correlation case, our algorithm, FilteredVamana, Post-Filtering, and NaviX perform almost equally well. Interestingly, Post-Filtering performs poorly on other subset filter cases but is extremely fast in this scenario. This is because the way we select query filters causes significant overlap between the query’s neighborhood and the filtered subset—meaning the points closest to the query vector are highly likely to satisfy the filter. For the negative correlation case, our algorithm, NaviX, and UNG are able to achieve > 0.95 recall, while our algorithm has the highest QPS.

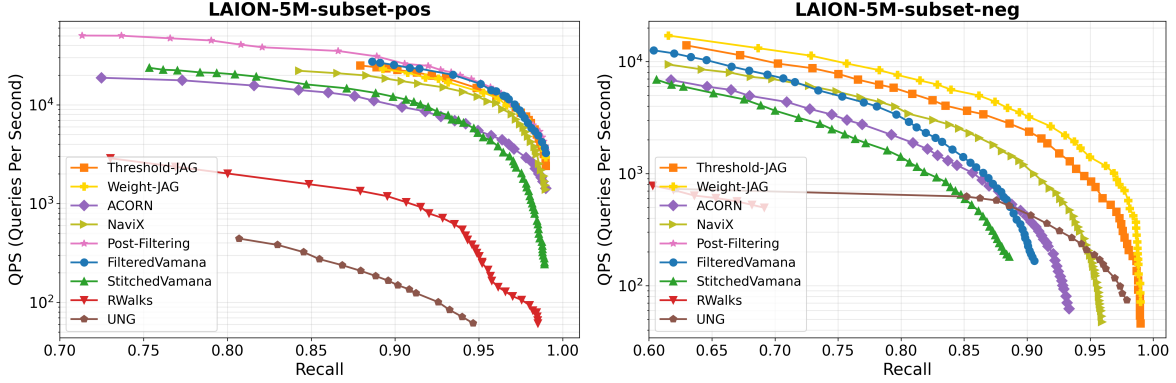


Figure 6. QPS vs. recall plots for subset filters with positive and negative correlations on the LAION-5M dataset.

Varying Filter Selectivity. We evaluate how the algorithms perform under different query selectivities. We conduct the experiment on MSTuring-10M using range filters and separately measure the recall of each algorithm when the query filter selectivities are at $1, 10^{-1}, 10^{-2}, 10^{-3}, 10^{-4}, 10^{-5}$. For each selectivity level, we report the maximum recall achieved while maintaining QPS > 5000 .

As shown in Figure 8, all general filtered vector search algorithms perform well only at high selectivity, and their recall gradually drops to zero as selectivity decreases. In contrast, our JAG algorithm is comparable to iRangeGraph, which is specifically designed for range filters.

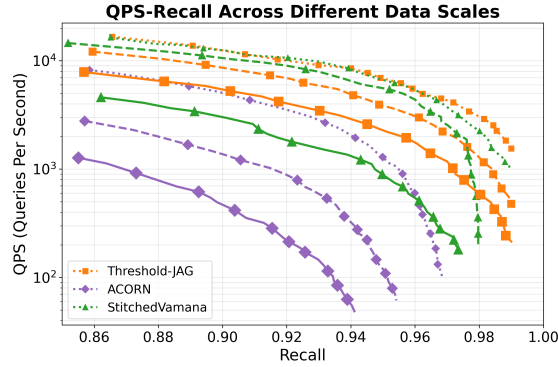


Figure 7. QPS vs. recall for subset filters on the LAION dataset. Each algorithm is represented by a different color; different line styles denote different dataset sizes: dotted for 1M, dashed for 5M, and solid for 25M.

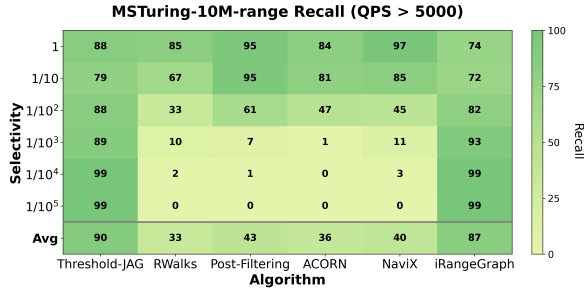


Figure 8. Recall for range filters on the MSTuring-10M dataset with different filter selectivities when maintaining QPS > 5000 .

C. Ablation Studies

Multiple thresholds. We perform an ablation study on MSTuring-10M for range filters and on LAION-5M for subset filters to examine how different thresholds help our algorithm handle filters with varying selectivity. Note that the original query filters in Section 4 consist of mixed selectivities. Here, we evaluate the performance of building our algorithm with different single thresholds and measure how they behave under filters of varying selectivity levels.

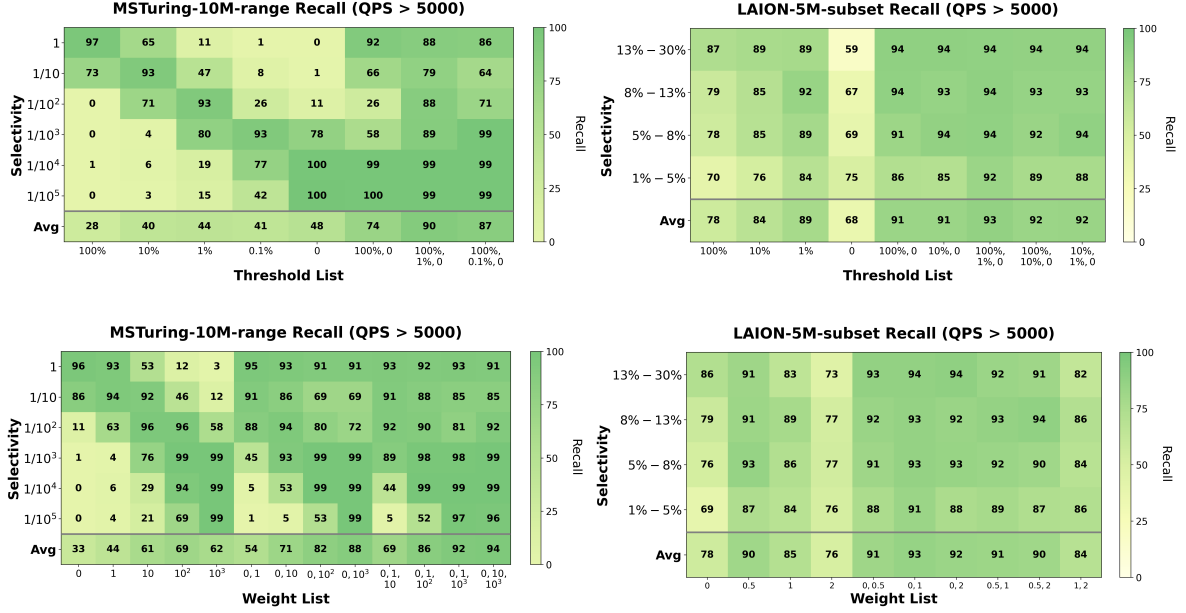


Figure 9. Query recall for different filter selectivities using different thresholds on the MSTuring-10M dataset with range filters (upper left) and the LAION-5M dataset with subset filters (upper right), and using different weights on the MSTuring-10M dataset with range filters (lower left) and the LAION-5M dataset with subset filters (lower right)

For MSTuring-10M-range, we consider six query selectivities: $1, 10^{-1}, 10^{-2}, 10^{-3}, 10^{-4}, 10^{-5}$. For LAION-5M-subset, we sort all query selectivities and evenly divide them into four ranges (13% – 30%, 8% – 13%, 5% – 8%, 1% – 5%). We use the same degree limit for all threshold choices. Please refer to the two upper plots in Figure 9 for the results.

When the single filter threshold is set to 0%—meaning we strictly compare by filter distance and only use standard distance when the filter distance is equal—the resulting index achieves perfect recall in low-selectivity cases (for both filter types). However, its recall gradually decreases as the selectivity increases. This is because, as selectivity weakens, the filter becomes less important, yet the query comparison still overemphasizes filter distance.

The opposite occurs with a 100% threshold, which is equivalent to searching purely by standard distance. This index performs well in high-selectivity cases but worse as the selectivity becomes more challenging. Interestingly, intermediate thresholds each perform best within a specific range of the selectivity spectrum and degrade as the selectivity deviates from their “comfort zone.” For example, in the subset-filter case, the 1% threshold performs best on queries whose selectivities are between 8% – 13% (i.e., the top 25–50% after ordering queries by selectivity). Its performance drops when the selectivity becomes either higher or lower. Similarly, in the range case, the 1% threshold performs best at $1/100$ and decreases in both directions.

By merging three different thresholds $\{100\%, 1\%, 0\}$ (sharing the same degree bound), the final index achieves strong and stable performance across all query selectivity levels.

Weight-JAG We also implement a weighted variant of JAG, in which attribute distance and vector distance are combined using different weights. For a weight w , we define

$$D_A^w(u, v) = w \cdot \text{dist}_A(a_u, a_v) + \text{dist}(x_u, x_v).$$

We build the index graph using a set of such weights. Similar to Threshold-JAG, when calling Insert and JointRobustPrune, we iterate over the weight list and use $D_A^w(u, v)$ as the comparator.

During query processing, we still compare $\text{dist}_F(\cdot)$ first, followed by the standard vector distance. We refer to this variant as Weight-JAG.

Multiple weights. We also conduct an ablation study on Weight-JAG to examine how different weights help handle filters with varying selectivities. From the lower two plots in Figure 9, we observe that Weight 0, which corresponds to building the proximity graph purely using the standard vector distance, achieves the best recall for standard nearest neighbor queries (selectivity 1). As the weight increases, the resulting proximity graph becomes more tailored to low-selectivity queries. By

Table 1. Dataset sizes and average selectivities. QPS (queries per second) and DC (distance computations) for Pre-Filtering algorithm.

		SIFT1M Label	ARXIV Label	ARXIV Range	LAION-1M Subset	LAION-5M Subset	LAION-25M Subset	YFCC Subset	MSTuring10M Subset	MSTuring10M Range	MSTuring10M Bool
Dataset	Size	10^6	$2.7 \cdot 10^6$	$2.7 \cdot 10^6$	10^6	$5 \cdot 10^6$	$2.5 \cdot 10^7$	10^7	10^7	10^7	10^7
	Avg Selectivity	0.083	0.37	0.43	0.099	0.098	0.099	0.018	0.15	0.17	0.15
Pre Filtering	QPS	412	5.4	4.6	146	28	5.6	5.2	27	31	2.3
	DC	$8.3 \cdot 10^4$	$1.0 \cdot 10^6$	$1.1 \cdot 10^6$	$9.9 \cdot 10^4$	$4.8 \cdot 10^5$	$2.4 \cdot 10^6$	$1.8 \cdot 10^5$	$1.5 \cdot 10^6$	$1.7 \cdot 10^6$	$1.5 \cdot 10^6$

merging graphs constructed with different weights, the final index achieves strong recall across all query selectivity levels.

Comparing Threshold-JAG and Weight-JAG. Though both implementations of our JAG (Threshold and Weight) can handle general filter search with different selectivities, they have different advantages. The Weight-JAG may have slightly better recall performance on MSTuring-10M-range, but its weight choices are not quite robust. For example, the optimal weight combination on MSTuring-10M-range is $\{0, 10, 1000\}$, but the optimal combination for LAION-5M-subset is $\{0, 2\}$. On the contrary, for Threshold-JAG, we choose the thresholds from the a subset of $\{100\%, 10\%, 1\%, 0.1\%, 0\}$ throughout our experiment.

To summarize our experimental findings on these two variations on our algorithm, we find that Threshold-JAG performs consistently well across different datasets and requires less tuning. On the other hand, Weight-JAG occasionally obtains the best results by a small margin; however, it is less robust across different datasets and requires more tuning. Our recommendation is to use Threshold-JAG as a robust and good-quality default.

Unifying multiple graphs. A natural question to ask is: *if the query selectivity were known in advance, could we simply build multiple independent indices, each tailored to a specific selectivity range, and route each query to the corresponding index?* Our results suggest that this strategy would be suboptimal.

From the LAION-5M experiment in Figure 9, we observe that for the queries with selectivities from 1% – 5%, the threshold combination $\{100\%, 1\%, 0\}$ outperforms *every* individual threshold. Similarly, for Weight-JAG, the weight combination $\{0, 2\}$ outperforms any individual weight configuration.

These findings indicate that greedy search benefits from access to edges constructed under *multiple* thresholds or weights. The search is able to adaptively traverse these heterogeneous edges to reach the target efficiently—something that a single, selectivity-specific index cannot provide.

D. Experimental Setup and Details

D.1. Machines

All experiments are conducted on a Google Cloud VM with the following specifications: an n2-highmem-64 instance equipped with 32 visible Intel Ice Lake vCPUs (1 vCPU per physical core) and 512 GB RAM. We run our experiments using 32 threads.

D.2. Additional Dataset Details

SIFT. The SIFT dataset (Lowe, 2004; Jegou et al., 2008) is one of the standard benchmarks for approximate nearest neighbor (ANN) and filtered-ANN research. It consists of one million points in $\mathbb{R}^{128}$. Following (Patel et al., 2024; Cai et al., 2024), we assign each point an integer attribute uniformly sampled from $\{1, \dots, 12\}$, and each query is assigned a label filter. A point satisfies the filter if its attribute matches the query’s label.

ARXIV. We use the dataset and filter setup from the recent benchmark in (Iff et al., 2025), which contains text embeddings of 2.7M arXiv papers generated using the `stella_en_400M_v5` model (Zhang et al., 2024). We apply both range and label filters: for range filters, the attribute is the publication date, and the query specifies a target time range; for label filters, the attribute is the number of subcategories assigned to each paper, and the query retrieves neighbors with a given number of subcategories.

LAION. We use the LAION dataset adopted in (Patel et al., 2024; Cai et al., 2024), which consists of CLIP embeddings of web-scraped images. Following the setup in (Patel et al., 2024), we construct a vocabulary of 30 common keywords

and assign to each image the three keywords whose embeddings are closest to that image vector. Each query specifies one keyword as the filter, and the search retrieves image vectors whose assigned keyword sets contain the specified term. We instantiate this setup at three scales—1M, 5M, and 25M points. Because the keywords capture the semantic content of images, we vary the correlation between filter and vector similarity by selecting different types of query filters: *positive* (the keyword closest to the query image), *random* (a randomly chosen keyword), and *negative* (the keyword farthest from the query image embedding).

YFCC. The YFCC dataset is the standard filtered-ANN benchmark used in the filtered search track of the NeurIPS 2023 competition. It consists of CLIP embeddings of 10M randomly selected images from YFCC100M (Flickr) (Thomee et al., 2016). Each image is associated with a “bag of tags” extracted from metadata such as the description text, camera model, year, and country, totaling over 200K unique attributes. Each query specifies a set of tags as the filter, and the goal is to retrieve images whose tag sets contain all of the query’s filter terms.

MSTuring. The MSTuring dataset consists of Bing search queries and corresponding answers. We use a 10M-point subset with 100-dimensional embeddings and synthetically construct *subset*, *range*, and *boolean* filters with mixed selectivity.

Subset filters: we define 30 binary attributes; each point independently includes each attribute with probability $1/2$. For a query, we randomly select $k \in \{0, 2, 4, 6, 8, 10, 12, 14, 16\}$ attributes and require that candidate points contain all selected attributes.

Range filters: each point is assigned an integer attribute in $[0, 10^6]$, and queries specify random intervals of length $10^6/k$ for $k \in \{1, 10, 100, 1000, 10^4, 10^5\}$ to control selectivity.

Boolean filters: Each query filter is a random Boolean function f over 15 Boolean variables, and each point’s attribute is a random instantiation x of these variables. A point satisfies the filter if $f(x) = \text{true}$. We control the selectivity by generating functions whose pass rates fall into four ranges: $(1/2^4, 1)$, $(1/2^8, 1/2^4)$, $(1/2^{12}, 1/2^8)$, and $(0, 1/2^{12})$.

D.3. Implementation Details for JAG

For Threshold-JAG, in practice, we determine the thresholds of Threshold-JAG by selecting the threshold values at specific points. For each point p with attribute a_p , we sample a set V of points (e.g., $|V| = 500$), compute the distribution of $\text{dist}_A(a_p, a_V)$, and consider candidate thresholds from the set of quantiles $\{100\%, 10\%, 1\%, 0\%\}$. We then choose the threshold list that yields the best performance.

For Weight-JAG, in practice, for each point p with attribute a_p , we sample a set V of points, compute the standard deviation σ_A of the attribute distance $\text{dist}_A(a_p, a_V)$ and the standard deviation σ of the vector distance $\text{dist}(x_p, x_V)$. Let $h = \sigma/\sigma_A$. We then select weights from the set

$$\{0, h, 2h, 5h, 10h, 100h, 1000h\}$$

and choose the subset that yields the best performance. Note that, compared to Threshold-JAG, the weight range used in Weight-JAG is much larger, and some extreme values (e.g., $1000h$) turn out to be useful in our experiments. This indicates that Weight-JAG is less robust than Threshold-JAG. Since using weighted combinations of attribute and vector distances has been explored in prior work (Wang et al., 2022; Ait Aomar et al., 2025), we include this variant primarily for ablation studies. For both performance and robustness, we recommend the Threshold-JAG version.

On Line 8 of Algorithm 4, we let V'_t include v if $v \in V'$ was already added by a previous threshold t . This is done to allow better pruning and improved edge utilization.

To accelerate the index building process without triggering too many JointRobustPrune calls, we introduce an early exit condition on Line 10 of Algorithm 4 where we terminate when $|V'_t| \geq 0.9 \cdot \deg/|T|$. This means we only store 90% of the degree limit after JointRobustPrune, with the expectation that this will prevent JointRobustPrune from being triggered every time a new edge is added. This is a common technique used in previous implementations.

For the YFCC and LAION datasets, we adopt a modification to the attribute distance design due to the high variance in their label distributions. For example, on YFCC, this variance is characterized by two factors:

1. The dataset contains $L = 200363$ distinct labels, and a single data point can be associated with between 0 and 1517 labels.
2. There is a large disparity in attribute frequency: some labels are shared across more than 30% of the dataset, while

Table 2. Compatibility of algorithms with different filter types. ✓ indicates support and ✗ indicates lack of support.

Filter Type	JAG	Post Filtering	ACORN	NaviX	RWalks ³	Filtered Vamana	Stitched Vamana	NHQ	UNG	iRange Graph
Label	✓	✓	✓	✓	✓	✓	✓	✓	✓	✗
Range	✓	✓	✓	✓	✓	✗	✗	✗	✗	✓
Subset	✓	✓	✓	✓	✓	✓	✓	✗	✓	✗
Boolean	✓	✓	✓	✓	✓	✗	✗	✗	✗	✗

others appear only once.

To mitigate this imbalance and prioritize less frequent attributes, we gather the frequency p_i for each attribute $i \in [L]$ and weight them by $\log(1/p_i)$. We then define the attribute distance as:

$$\text{dist}_A(a_u, a_v) = C - \sum_{i \in a_u \cap a_v} \log(1/p_i)$$

for a sufficiently large constant C . This design prioritizes filters appearing less frequently in the dataset and focuses on the shared labels, rather than the differences between them.

D.4. Additional Algorithm Details

Below, we briefly describe each baseline algorithm we evaluate and how we choose its parameters.

Specifically, if an algorithm and dataset were previously evaluated in the original paper, we directly adopt their parameter settings. Otherwise, we refer to the parameter options recommended in the original paper and select the configuration that achieves the highest QPS at recall 0.95. If the optimal recall the algorithm achieves at QPS=50 is less than 0.95 we then pick the configuration that achieves the highest possible recall.

ACORN. (Patel et al., 2024) proposes two modified variants, ACORN-1 and ACORN- γ . We focus on ACORN- γ in our experiments because it achieves better search quality. ACORN- γ constructs a denser graph and applies two-hop expansion during search. Suppose the minimum filter selectivity is γ and M is the degree parameter used in standard HNSW. ACORN- γ uses two-hop expansion to ensure that each node has access to approximately $M \cdot \gamma$ nearby points. After filtering out the points that do not satisfy the query filter, the remaining neighbors should approximate those that would be obtained by building a standard HNSW graph solely on the points satisfying the filter.

For the SIFT and LAION datasets used in the ACORN paper, we adopt the parameter settings suggested by the authors. For the other datasets, we select parameters from the set $\{M = 32, M_\beta = 32\}$, $\{M = 32, M_\beta = 64\}$, and $\{M = 64, M_\beta = 64\}$. For γ , we set it to the minimum of the 5% most selective queries and 30.

NaviX. (Sehgal & Salihoglu, 2025) proposes an improved search heuristic on top of ACORN. It constructs a standard HNSW graph, and at query time adaptively decides whether to expand one-hop or two-hop neighbors based on local selectivity estimates. We use the authors’ FAISS-Navix implementation with the *adaptive-local* heuristic enabled. The parameter configurations we test are $\{M = 32, \text{efc} = 200\}$ and $\{M = 64, \text{efc} = 200\}$, and we choose the better-performing configuration for each dataset.

FilteredVamana. (Gollapudi et al., 2023) proposes an extended version of the Vamana algorithm tailored to subset-filtered search. The original experiments focus on the single-filter case, which naturally generalizes to multiple OR-filters. In our evaluation, we also test the algorithm on multiple AND-filters (our subset setting). The key idea is that when inserting a point, the algorithm traverses and connects only to those existing points that share at least one common attribute with the inserted point. During their “FilteredRobustPruning”, if an edge (p, p') is pruned in favor of (p, p^*) , the replacement point p^* must cover all the attributes shared between p and p' . At query time, the algorithm restricts traversal to points that satisfy the query filter. We select optimal parameters from, $\text{deg} \in \{64, 96, 128\}$ and $\alpha = 1.2$.

³The original filter distance used in RWalks only supports binary match/non-match filters (e.g., label and subset filters). In our experiments, we implement RWalks with our proposed filter distance, which generalizes to a much broader range of filter types and yields uniformly improved performance.

StitchedVamana. This is the second algorithm introduced in (Gollapudi et al., 2023). It is primarily designed for the single-filter setting, but we also evaluate it under multiple AND-filters (our subset case). For each distinct label in the dataset, the algorithm builds a graph over the points containing that label using a small degree parameter R_{small} . For points with multiple labels, it merges the outgoing edges from the corresponding per-label graphs using *FilteredRobustPruning* with a larger degree R_{stitched} . During query processing, traversal is restricted to points satisfying the filter. The parameter configurations we test are $R_{\text{stitched}} \in \{64, 96, 128\}$, $R_{\text{small}} = R_{\text{stitched}}/2$, and $L = 1.5 \cdot R_{\text{stitched}}$.

RWalks. (Ait Aomar et al., 2025) builds on a standard unfiltered index. For each point, the algorithm performs a random walk to compute an aggregated attribute vector. When a query arrives, it defines the filter distance between a point and the query as the fraction of labels in the aggregated attribute vector that match the query filter. In our experiments, we implement RWalks using our proposed filter distance, which strictly generalizes the binary match/non-match criterion used in the original paper and yields uniformly improved performance. During search, the greedy traversal is guided by a weighted combination of the filter distance and the standard vector distance, using a weight parameter h . The hyperparameter h is reported to work best when set to 0.1 (after normalization).

We implement the algorithmic ideas of RWalks on top of Vamana and introduce minor modifications to improve performance (e.g., we use our generalized attribute and filter distance definitions instead of their binary equality-based definition). The degree parameters we test are $\text{deg} \in \{64, 96, 128\}$. For all other parameters, we adopt the values recommended in the original paper: $m = 5$, $d = 3$, $\tau = 0$, and $h = 0.1$.

Post-Filtering. We first retrieve the nearest neighbors without applying any filter constraints, and then perform filter checks on the results to obtain the first k points that satisfy the query filter. We test degree parameters $\text{deg} \in \{32, 64, 96, 128\}$.

Pre-Filtering. We first apply the filter constraints to the points, and then compute their distances to obtain the top- k results. No index is required for this Pre-Filtering method.

NHQ. (Wang et al., 2022) proposes the NHQ algorithm, which targets label-equality filters. It defines an attribute distance between two points based solely on whether their attributes are equal, and combines this with the standard vector distance using a weighted average. Because this attribute distance can only distinguish equality, NHQ can handle only equality-based label filters in our experiments. We use the parameter settings recommended in (Iff et al., 2025) for both the ARXIV and SIFT datasets.

UNG. (Cai et al., 2024) introduces the Unified Navigating Graph (UNG), designed specifically for subset-filter queries. The algorithm first constructs a proximity graph for each possible label set. It then builds a label-navigating graph over the label sets by connecting a directed edge whenever one label set subsumes another. Given a subset query, UNG maintains a trie data structure to identify feasible starting points, and then performs greedy search from these entry points. We primarily adopt the recommended parameters from the original paper: $\alpha = 1.2$, $\delta = 6$, and $\sigma = 16$. For the degree R and queue length L , we choose the best-performing configuration from $(R, L) \in \{(32, 100), (48, 150), (64, 200)\}$.

iRangeGraph. (Xu et al., 2024) proposes iRangeGraph for range-filter queries. It constructs a segment tree to decompose the attribute range into intervals of different lengths, builds a graph index for each interval, and searches only over those interval graphs that overlap with the query range. We use the parameter settings recommended by the authors: $m = 16$ and $\text{EF} = 100$.

D.5. Parameter Settings

We summarize the parameters used for each algorithm and dataset below.

- **Threshold-JAG.** We use $\alpha = 1.2$ for all datasets.
 - **SIFT-1M (label):** $\text{deg} = 96$, Thresholds = $\{10\%, 0\}$
 - **ARXIV (label):** $\text{deg} = 64$, Thresholds = $\{100\%, 0\}$
 - **ARXIV (range):** $\text{deg} = 96$, Thresholds = $\{100\%, 10\%\}$
 - **LAION-1M / LAION-5M / LAION-25M:** $\text{deg} = 96$, Thresholds = $\{10\%, 1\%, 0\}$
 - **YFCC:** $\text{deg} = 128$, Thresholds = $\{10\%, 1\%, 0\}$
 - **MSTuring (subset, range, boolean):** $\text{deg} = 128$, Thresholds = $\{100\%, 1\%, 0\}$

- Weight-JAG. We use $\alpha = 1.2$ for all datasets.
 - **SIFT-1M (label):** deg = 96, Weight = $\{1\}$
 - **ARXIV (label):** deg = 64, Weight = $\{0, 10\}$
 - **ARXIV (range):** deg = 96, Weight = $\{0, 2\}$
 - **LAION-1M / LAION-5M / LAION-25M (subset):** deg = 96, Weight = $\{0, 1\}$
 - **YFCC (subset):** deg = 128, Weight = $\{0, 1\}$
 - **MSTuring (subset):** deg = 128, Weight = $\{0, 1, 5\}$
 - **MSTuring (range):** deg = 128, Weight = $\{0, 10, 1000\}$
 - **MSTuring (boolean):** deg = 128, Weight = $\{0, 1, 10\}$
- ACORN.
 - **SIFT-1M (label):** $M = 32, \gamma = 12, M_\beta = 64$
 - **ARXIV (label):** $M = 32, \gamma = 25, M_\beta = 32$
 - **ARXIV (range):** $M = 32, \gamma = 10, M_\beta = 32$
 - **LAION-1M / LAION-5M / LAION-25M (subset):** $M = 32, \gamma = 30, M_\beta = 32$
 - **YFCC (subset):** $M = 64, \gamma = 30, M_\beta = 64$
 - **MSTuring (subset):** $M = 64, \gamma = 30, M_\beta = 64$
 - **MSTuring (range):** $M = 64, \gamma = 30, M_\beta = 64$
 - **MSTuring (boolean):** $M = 32, \gamma = 30, M_\beta = 64$
- NaviX.
 - **SIFT-1M (label):** $M = 32, efc = 200$
 - **ARXIV (label):** $M = 64, efc = 200$
 - **ARXIV (range):** $M = 64, efc = 200$
 - **LAION-1M / LAION-5M / LAION-25M (subset):** $M = 32, efc = 200$
 - **YFCC (subset):** $M = 64, efc = 200$
 - **MSTuring (subset):** $M = 64, efc = 200$
 - **MSTuring (range):** $M = 64, efc = 200$
 - **MSTuring (boolean):** $M = 64, efc = 200$
- RWalks. $\alpha = 1.2, m = 5, d = 3, \tau = 0, h = 0.1$
 - **SIFT-1M (label):** deg = 64
 - **ARXIV (label):** deg = 64
 - **ARXIV (range):** deg = 64
 - **LAION-1M (subset) :** deg = 64
 - **LAION-5M / LAION-25M (subset):** deg = 96
 - **YFCC (subset):** deg = 96
 - **MSTuring (subset):** deg = 96
 - **MSTuring (range):** deg = 128
 - **MSTuring (boolean):** deg = 96
- Post-Filtering. $\alpha = 1.2$
 - **SIFT-1M (label):** deg = 32
 - **ARXIV (label):** deg = 96
 - **ARXIV (range):** deg = 32
 - **LAION-1M (subset) :** deg = 32
 - **LAION-5M (subset):** deg = 64
 - **LAION-25M (subset):** deg = 128

- **YFCC (subset):** deg = 64
- **MSTuring (subset):** deg = 96
- **MSTuring (range):** deg = 64
- **MSTuring (boolean):** deg = 64
- FilteredVamana. $\alpha = 1.2$
 - **SIFT-1M (label):** deg = 64
 - **ARXIV (label):** deg = 96
 - **LAION-1M (subset) :** deg = 96
 - **LAION-5M (subset):** deg = 96
 - **LAION-25M (subset):** deg = 128
 - **YFCC (subset):** deg = 128
 - **MSTuring (subset):** deg = 128
- StitchedVamana. $\alpha = 1.2$
 - **SIFT-1M (label):** deg = 64
 - **ARXIV (label):** deg = 96
 - **LAION-1M (subset) :** deg = 96
 - **LAION-5M (subset):** deg = 96
 - **LAION-25M (subset):** deg = 96
 - **YFCC (subset):** deg = 64
 - **MSTuring (subset):** deg = 128
- NHQ
 - **SIFT-1M (label):** $K = 80, L = 60, S = 10, R = 200, \text{RANGE} = 60, PL = 300, B = 0.6, \text{weight_search} = 1000000$
 - **ARXIV (label):** $K = 80, L = 60, S = 10, R = 200, \text{RANGE} = 60, PL = 300, B = 0.6, \text{weight_search} = 1000000$
- UNG. $\alpha = 1.2, \delta = 6, \sigma = 16$
 - **SIFT-1M (label):** deg = 32, L = 100
 - **ARXIV (label):** deg = 48, L = 150
 - **LAION-1M (subset):** deg = 32, L = 100
 - **LAION-5M (subset):** deg = 32, L = 100
 - **LAION-25M (subset):** deg = 32, L = 100
 - **YFCC (subset):** deg = 32, L = 100
- iRangeGraph.
 - **ARXIV (range):** $M = 16, ef = 100$
 - **MSTuring (range):** $M = 16, ef = 100$

D.6. Indexing Time

We report the indexing time in Table 3 for all algorithms and datasets using the parameters specified in Section D.5. As described in Section 4.2, our parameter-selection guideline is to maximize QPS at 0.95 recall, rather than to optimize indexing time or index size.

D.7. Distance Computation v.s. Recall

We report recall@10 versus distance computation in Figure 10, 11, 12, 13.

Table 3. Indexing time for all algorithms across different datasets. We show the indexing time for Threshold-JAG; the indexing time for Weight-JAG is similar. Please refer to Appendix D.5 for the parameters used to build the index.

	JAG	ACORN	NaviX	Filtered Vamana	Stitched Vamana	Post Filtering	RWalks	UNG	NHQ	iRange Graph
SIFT-1M-label	100s	222s	54s	24s	26s	11s	122s	15s	28s	N/A
ARXIV-2M-label	3174s	16016s	2378s	3011s	5599s	3409s	19685s	3779s	515s	N/A
ARXIV-2M-range	5978s	5211s	2081s	N/A	N/A	750s	19642s	N/A	N/A	7424s
LAION-1M-subset	261s	1148s	145s	211s	1249s	31s	365s	13s	N/A	N/A
LAION-5M-subset	1537s	6960s	872s	1232s	3683s	328s	3293s	89s	N/A	N/A
LAION-25M-subset	8687s	45662s	4981s	9401s	36179s	3209s	17733s	569s	N/A	N/A
YFCC-10M-subset	7898s	22572s	1220s	4515s	42812s	300s	3355s	8453s	N/A	N/A
MSTuring-10M-subset	3760s	21742s	1276s	1257s	33376s	500s	4140s	>15h	N/A	N/A
MSTuring-10M-range	3348s	21322s	1281s	N/A	N/A	311s	7228s	N/A	N/A	6596s
MSTuring-10M-bool	3760s	21185s	1445s	N/A	N/A	310s	4140s	N/A	N/A	N/A

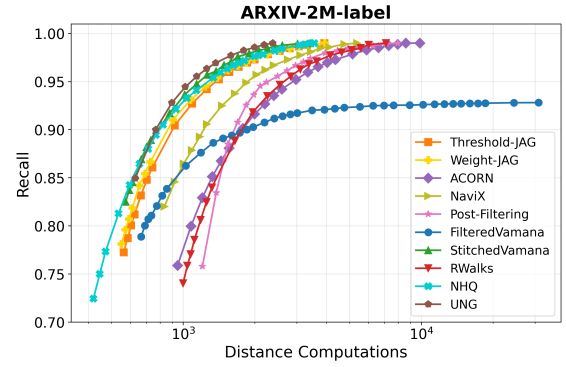
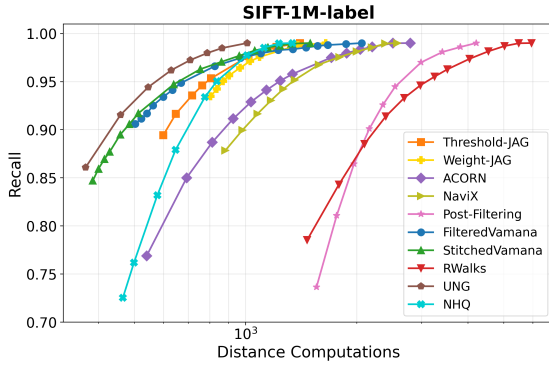


Figure 10. Distance computation vs. recall plot for label filters on the SIFT and ARXIV datasets. Note that NHQ is designed specifically for Label filter. FilteredVamana, StitchedVamana, and UNG are designed specifically for Label and Subset filters.

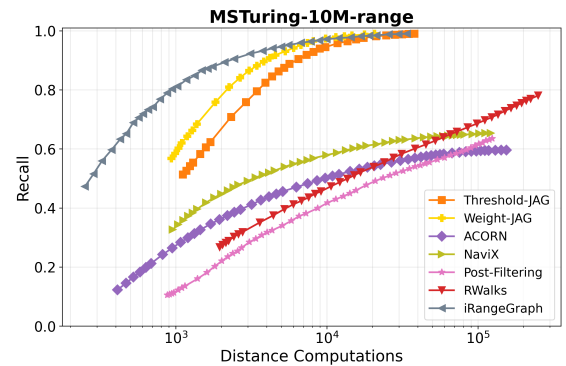
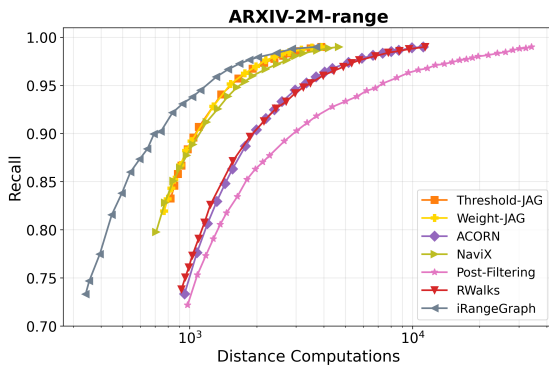


Figure 11. Distance computation vs. recall plot for range filters on the ARXIV and MSTuring datasets. Note that iRangeGraph is designed specifically for Range filter

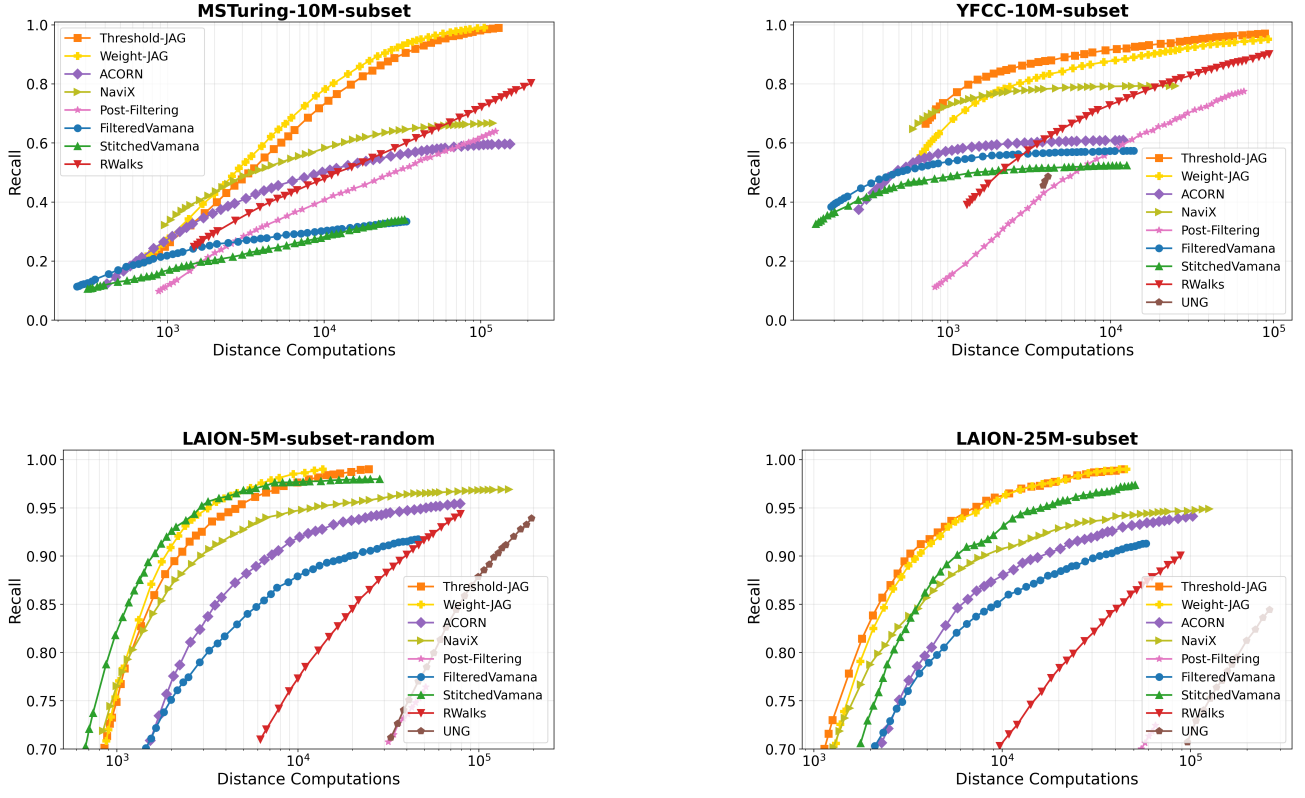


Figure 12. Distance computation vs. recall plots for subset filters on the MSTuring-10M, LAION-5M, LAION-25M, and YFCC dataset. Note that FilteredVamana, StitchedVamana, and UNG are designed specifically for label and subset filters.

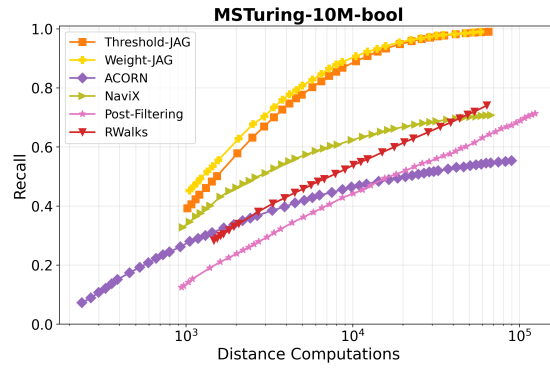


Figure 13. Distance computation vs. recall for boolean filters on the MSTuring dataset.